

MCDI500

PROGRAMACIÓN PARA LA

CIENCIA DE DATOS

AVANCE DEL PROYECTO

FASE 3

Título del proyecto: Impacto de la inteligencia artificial en el empleo, la productividad y la capacitación laboral “Sumativa Fase 3 – Semana 2”

Integrantes:

Pablo Rodríguez

Luiskar Espinoza

Curso: MCDI500 — PROGRAMACIÓN PARA LA CIENCIA DE DATOS

Docente: Dr. Omar Salinas Silva.

Fecha: 14/06/2026

Indice

Indice	2
II.Diseño de soluciones algorítmicas eficientes	3
II.a Codificación funcional y arquitectura básica del script	3
II.b Preprocesamiento y transformación del dataset.....	5
II.c Validación técnica y verificación del código	7
II.e Diseño estructurado del código	10
III. Implementación de código modular y robusto	13
III.a Programación orientada a objetos	13
IV. Repositorio GitHub (F3)	22
V. Notebooks ejecutables (F3).....	26
VI. Conclusiones	30
VII. Bibliografía.....	31

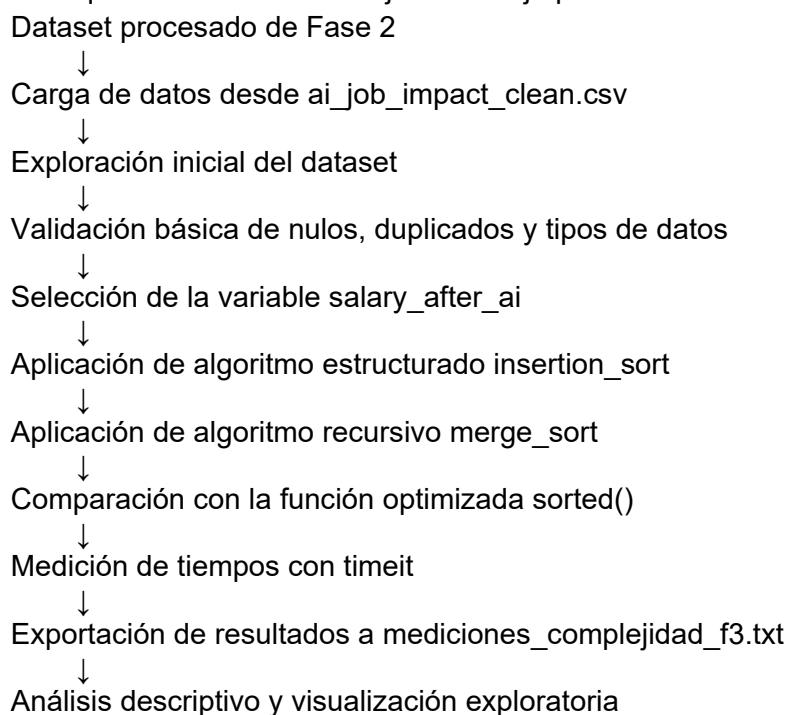
II. Diseño de soluciones algorítmicas eficientes

II.a Codificación funcional y arquitectura básica del script

La codificación funcional desarrollada en la Fase 3 se estructuró con el objetivo de implementar scripts operativos, claros y verificables, capaces de integrar el dataset procesado en la Fase 2 con algoritmos de ordenamiento, mediciones de eficiencia y componentes iniciales de Programación Orientada a Objetos.

El desarrollo se encuentra organizado principalmente en dos componentes técnicos: el notebook `F3_algoritmos_complejidad.ipynb` y el script `algoritmos_f3.py`. El notebook funciona como entorno de ejecución, documentación e interpretación de resultados, mientras que el script contiene clases orientadas a objetos utilizadas para administrar y analizar el dataset.

La arquitectura básica del flujo de trabajo puede resumirse de la siguiente manera:



Esta organización permite mantener coherencia entre las entradas, el procesamiento y las salidas. La entrada principal corresponde al archivo `ai_job_impact_clean.csv`, generado en la Fase 2. El procesamiento se realiza mediante funciones y clases implementadas en Python, y las salidas corresponden a resultados visibles en el notebook, mediciones temporales, tablas descriptivas y archivos exportados a la carpeta `F3/results`.

Desde el punto de vista de programación estructurada, se implementó el algoritmo `insertion_sort`, el cual permite ordenar una lista de valores utilizando ciclos, condicionales y desplazamientos sucesivos. Este algoritmo recibe como parámetro una lista de salarios y devuelve una nueva lista ordenada. Su implementación permite representar una solución clara, secuencial y verificable.

También se incorporó el algoritmo recursivo `merge_sort`, basado en la estrategia de dividir el problema en partes más pequeñas. Este algoritmo utiliza un caso base para detener la recursión y un caso recursivo para dividir la lista en dos subconjuntos. Luego, la función auxiliar `combinar_listas` integra ambas partes en una lista final ordenada. Esta implementación permite evidenciar el uso de recursividad y de una lógica de división funcional aplicada a datos reales del proyecto.

Además de los algoritmos manuales, se utilizó la función `sorted()` de Python como referencia optimizada. Esta comparación permitió contrastar el rendimiento de implementaciones propias frente a una función incorporada en el lenguaje, lo que fortalece el análisis de eficiencia desarrollado posteriormente.

En el script `algoritmos_f3.py` se implementaron las clases `PreprocesadorAIJob` y `PreprocesadorAIJobAvanzado`. La primera clase administra la carga, exploración, limpieza, validación y resumen del dataset. La segunda clase hereda de la clase base e incorpora métodos adicionales para calcular estadísticas de salario y filtrar información por riesgo de automatización o industria. Esta separación permite avanzar desde un notebook puramente procedimental hacia una estructura más modular y reutilizable.

Entre los métodos implementados se encuentran:

Componente	Responsabilidad
cargar()	Leer el dataset desde un archivo CSV.
explorar()	Mostrar primeros registros, información general y valores nulos.
limpiar()	Eliminar registros con valores nulos cuando corresponda.
validar()	Verificar que no existan valores nulos en el dataset.
obtener_salarios()	Extraer la variable <code>salary_after_ai</code> como lista de trabajo.
resumen()	Generar estadísticas descriptivas del dataset.
promedio_salario()	Calcular el salario promedio posterior a la adopción de IA.
filtrar_por_riesgo()	Filtrar registros según nivel de riesgo de automatización.
filtrar_por_industria()	Filtrar registros según industria.

El código mantiene un estilo claro y mantenible, apoyado en nombres descriptivos, funciones con parámetros definidos, docstrings y separación básica de responsabilidades. Cada bloque del notebook cumple una función específica dentro del flujo general: carga de datos, exploración, validación, aplicación de algoritmos, medición de tiempos, interpretación y exportación de resultados.

En síntesis, la codificación funcional y la arquitectura básica del script permiten evidenciar un flujo de datos coherente, verificable y alineado con los objetivos de la Fase 3. La solución integra programación estructurada, recursividad y una primera organización orientada a objetos, aportando una base técnica para el desarrollo del núcleo algorítmico del proyecto.

II.b Preprocesamiento y transformación del dataset

El preprocesamiento utilizado en la Fase 3 se desarrolló a partir del dataset procesado previamente durante la Fase 2, específicamente el archivo `ai_job_impact_clean.csv`, almacenado en la carpeta `data/processed` del repositorio. Esta decisión permite mantener continuidad metodológica entre las fases del proyecto, ya que la Fase 2 tuvo como foco principal la obtención, limpieza y transformación inicial de los datos, mientras que la Fase 3 se concentra en el diseño del núcleo algorítmico, la eficiencia computacional y la aplicación progresiva de principios de Programación Orientada a Objetos.

El dataset utilizado contiene 2.000 registros y 17 columnas relacionadas con empleo, adopción de inteligencia artificial, riesgo de automatización, experiencia laboral, salarios, satisfacción laboral, productividad y requerimientos de capacitación. Estas variables permiten vincular el trabajo algorítmico con la problemática general del proyecto: analizar el impacto de la inteligencia artificial en el empleo, la productividad y la capacitación laboral.

En el notebook `F3_algoritmos_complejidad.ipynb`, el dataset fue cargado mediante la librería Pandas desde la ruta definida para los datos procesados:

```
DATA_PROCESSED = PROJECT_ROOT / "data" / "processed" / "ai_job_impact_clean.csv"
df = pd.read_csv(DATA_PROCESSED)
```

La primera validación consistió en confirmar la correcta lectura del archivo, verificando la cantidad de filas y columnas. Esta comprobación permitió asegurar que el notebook estuviera trabajando sobre el dataset esperado y que la ruta definida dentro del proyecto fuera coherente con la estructura del repositorio. Posteriormente, se realizó una revisión de integridad del conjunto de datos mediante la evaluación de valores nulos, registros duplicados y tipos de datos. Para ello, se utilizaron instrucciones como:

```
df_algoritmico.isnull().sum()
df_algoritmico.duplicated().sum()
df_algoritmico.dtypes
```

Los resultados mostraron que las variables revisadas no presentaban valores nulos, que no existían registros duplicados y que los tipos de datos eran coherentes con el uso previsto en el análisis. En particular, las variables numéricas como `age`, `years_experience`, `salary_before_ai`, `salary_after_ai`, `work_hours_per_week`, `job_satisfaction` y `productivity_change_%` fueron identificadas como variables aptas para cálculos, ordenamientos y análisis descriptivo. Las variables categóricas, como `automation_risk`, `ai_adoption_level` y `upskilling_required`, se conservaron para análisis de filtrado e interpretación.

La variable principal seleccionada para el análisis algorítmico fue `salary_after_ai`, correspondiente al salario posterior a la adopción de inteligencia artificial. Esta variable fue elegida porque cumple con tres condiciones relevantes: es numérica, se relaciona directamente con la problemática del proyecto y permite aplicar algoritmos de ordenamiento y mediciones de eficiencia temporal.

Para utilizar esta variable dentro de los algoritmos manuales, se transformó desde una columna de Pandas a una lista de Python. Esta transformación fue necesaria porque los algoritmos `insertion_sort` y `merge_sort` fueron diseñados para recibir listas como estructura de entrada:

```
datos_prueba = df["salary_after_ai"].head(500).tolist()
```

La selección de los primeros 500 registros permitió trabajar con una muestra suficientemente amplia para comparar tiempos de ejecución, pero sin comprometer la estabilidad del notebook. Esta decisión fue especialmente relevante considerando que el algoritmo `insertion_sort` tiene una complejidad temporal esperada de $O(n^2)$, por lo que su tiempo de ejecución puede crecer significativamente al aumentar el tamaño de la entrada.

Además del uso de `salary_after_ai`, el notebook incorporó una revisión descriptiva de variables asociadas al análisis laboral y salarial. Mediante el método `resumen()` de la clase `PreprocesadorAIJob`, se obtuvieron estadísticas descriptivas de variables numéricas como edad, experiencia, salario antes de la IA, salario después de la IA, horas de trabajo, satisfacción laboral y cambio de productividad. Esta revisión permitió contextualizar el comportamiento general del dataset antes de aplicar los algoritmos.

También se incorporó una visualización exploratoria mediante un diagrama de caja comparativo entre `salary_before_ai` y `salary_after_ai`. Esta representación permitió observar la distribución de los salarios antes y después de la adopción de inteligencia artificial, identificando diferencias en tendencia central, dispersión y valores extremos. Aunque esta visualización no forma parte directa del algoritmo de ordenamiento, aporta contexto analítico para comprender la variable seleccionada y su relación con la pregunta del proyecto.

Desde el punto de vista de transformación de datos, en esta fase no se rehízo el pipeline completo de limpieza, casting o normalización, ya que el dataset provenía de la Fase 2, donde fue preparado para su uso analítico. Sin embargo, sí se aplicaron transformaciones específicas para el análisis algorítmico: selección de variables relevantes, conversión de columnas a listas, definición de subconjuntos de prueba y verificación de integridad antes de ejecutar los algoritmos.

La justificación técnica de este enfoque se basa en la separación de responsabilidades entre fases. La Fase 2 se encargó del procesamiento general del dataset, mientras que la Fase 3 reutiliza ese resultado para implementar algoritmos, medir eficiencia y organizar componentes mediante POO. Esta decisión evita duplicar procesos, mantiene la trazabilidad del proyecto y permite concentrar el desarrollo en el núcleo algorítmico solicitado.

En síntesis, el preprocesamiento de la Fase 3 permitió confirmar que el dataset procesado era íntegro, estable y apto para el análisis algorítmico. Se verificaron valores nulos, duplicados y tipos de datos; se seleccionó una variable numérica relevante; se transformó dicha variable en una lista para su uso en algoritmos estructurados y recursivos; y se complementó el análisis con estadísticas descriptivas y visualización exploratoria. Esta preparación permitió asegurar la coherencia entre los datos de entrada, el procesamiento aplicado y los resultados obtenidos en las mediciones de complejidad.

El uso de Pandas permite trabajar con estructuras tabulares de datos, facilitando operaciones de carga, validación, exploración y transformación dentro de flujos reproducibles de ciencia de datos (The pandas development team, 2026).

II.c Validación técnica y verificación del código

La validación técnica del código se realizó directamente en el notebook `F3_algoritmos_complejidad.ipynb`, mediante la ejecución secuencial de las celdas y la revisión de resultados intermedios generados durante el flujo de trabajo. El objetivo de esta validación fue comprobar que el dataset procesado pudiera ser cargado correctamente, que las variables utilizadas estuvieran disponibles y que los algoritmos implementados produjeran resultados coherentes con las entradas utilizadas.

En primer lugar, se verificó la correcta carga del archivo `ai_job_impact_clean.csv`, proveniente de la Fase 2. Para ello, se definieron las rutas del proyecto y se leyó el archivo mediante Pandas. La ejecución correcta de esta etapa permitió confirmar que el notebook se encontraba ubicado dentro de la estructura esperada del repositorio y que podía acceder al dataset procesado desde la carpeta `data/processed`.

Posteriormente, se revisó la estructura general del dataset. Esta validación incluyó la comprobación de dimensiones, nombres de columnas y tipos de datos. La salida del notebook confirmó que el dataset contenía 2.000 filas y 17 columnas, incluyendo variables relevantes para el análisis como `salary_after_ai`, `salary_before_ai`, `automation_risk`, `ai_adoption_level`, `years_experience`, `job_satisfaction` y `productivity_change_%`.

También se verificó la integridad básica del subconjunto de variables utilizadas en el análisis algorítmico. Para ello, se revisaron valores nulos mediante `isnull().sum()`, registros duplicados mediante `duplicated().sum()` y tipos de datos mediante `dtypes`. Esta revisión permitió confirmar que las variables seleccionadas no presentaban nulos, que no existían duplicados en el subconjunto revisado y que los tipos de datos eran consistentes con las operaciones realizadas.

La validación funcional de los algoritmos se realizó mediante pruebas con datos reales del dataset. En primer lugar, se extrajeron los primeros 20 valores de la variable `salary_after_ai` y se transformaron en una lista de Python. Esta lista fue utilizada como entrada para el algoritmo estructurado `insertion_sort`. La salida generada por el notebook permitió comparar los valores originales con los valores ordenados, comprobando visualmente que el algoritmo devolvía una lista organizada de menor a mayor.

De manera similar, se validó el funcionamiento del algoritmo recursivo `merge_sort`. Este algoritmo fue ejecutado sobre una lista de salarios proveniente de la misma variable `salary_after_ai`. La salida obtenida confirmó que la función recursiva ordenaba correctamente los valores, validando tanto el caso base como el proceso de división y combinación de listas. La función auxiliar `combinar_listas` permitió integrar los subconjuntos ordenados en una lista final coherente.

Además de las pruebas funcionales con 20 registros, se ejecutaron pruebas con una muestra de 500 registros para validar que los algoritmos pudieran operar sobre un volumen mayor de datos. Esta etapa permitió comprobar que `insertion_sort`, `merge_sort` y `sorted()` procesaban la misma entrada sin errores y entregaban resultados comparables para las mediciones de eficiencia temporal.

La validación también incluyó el uso de la clase `PreprocesadorAIJob`, definida en el script `algoritmos_f3.py`. Esta clase permitió cargar el dataset, explorarlo, limpiarlo, transformar columnas numéricas, validar ausencia de nulos, obtener la columna de salarios y generar un resumen estadístico. La clase

PreprocesadorAIJobAvanzado, que hereda de la clase base, incorporó métodos adicionales para calcular salario promedio, salario máximo, salario mínimo y filtrar registros por riesgo de automatización o industria. Esta estructura permitió verificar el uso funcional de clases y métodos en el flujo del proyecto.

Como evidencia adicional, el notebook generó resultados persistentes en el archivo mediciones_complejidad_f3.txt, ubicado en la carpeta F3/results. Esta exportación permitió registrar los tiempos de ejecución obtenidos y conservar una evidencia externa al notebook, fortaleciendo la trazabilidad del proceso.

Desde el punto de vista de casos de prueba, la validación actual no se limitó únicamente a la revisión exploratoria del dataset, sino que incorporó pruebas formales mediante assert. Estas pruebas permitieron verificar el comportamiento de los algoritmos `insertion_sort` y `merge_sort` frente a distintos escenarios: listas vacías, listas con un único elemento y listas pequeñas desordenadas. La ejecución correcta de estas pruebas confirmó que ambos algoritmos responden adecuadamente tanto en casos normales como en casos límite.

Además, se incorporó manejo básico de excepciones mediante bloques `try/except`. Esta validación permitió comprobar el comportamiento del código frente a entradas no válidas, controlando errores sin interrumpir la ejecución general del notebook. De esta forma, la validación técnica se fortaleció al incluir no solo revisión de datos y resultados intermedios, sino también pruebas funcionales y manejo controlado de errores.

En conjunto, estas validaciones entregan evidencia más robusta del funcionamiento del sistema, ya que permiten comprobar que los algoritmos producen resultados esperados bajo distintos escenarios y que el flujo de ejecución puede responder frente a situaciones no previstas. Por lo tanto, la validación técnica de la Fase 3 evidencia coherencia entre las entradas utilizadas, los algoritmos aplicados, los resultados obtenidos y las pruebas ejecutadas dentro del notebook.

II.d Eficiencia y optimización

La eficiencia y optimización del código fueron abordadas mediante la comparación de distintas estrategias de ordenamiento aplicadas sobre datos reales del proyecto. El objetivo de esta etapa fue evaluar el rendimiento de implementaciones algorítmicas diferentes, utilizando una medición reproducible que permitiera fundamentar técnicamente la elección de la solución más eficiente.

Para realizar las pruebas se utilizó la variable `salary_after_ai`, correspondiente al salario posterior a la adopción de inteligencia artificial. Esta variable fue seleccionada porque es numérica, se relaciona directamente con la problemática del proyecto y permite aplicar algoritmos de ordenamiento de manera clara y verificable. A partir de esta columna se construyó una muestra de 500 registros, transformada en una lista de Python para ser procesada por los algoritmos implementados.

Las implementaciones comparadas fueron las siguientes:

Implementación	Tipo de enfoque	Propósito dentro del análisis
<code>insertion_sort</code>	Algoritmo estructurado manual	Evidenciar una solución basada en ciclos, condicionales y comparaciones sucesivas.

merge_sort	Algoritmo recursivo	Aplicar una estrategia de división del problema en subproblemas más pequeños.
sorted()	Función optimizada de Python	Utilizar una referencia eficiente incorporada en el lenguaje para comparar desempeño.

La medición de tiempos se realizó mediante la librería `timeit`, la cual permite ejecutar pruebas reproducibles sobre fragmentos de código y comparar su tiempo de ejecución bajo condiciones similares. Esta herramienta fue utilizada para medir el rendimiento de cada implementación sobre la misma muestra de datos, evitando comparaciones subjetivas o basadas únicamente en apreciaciones teóricas.

Los resultados obtenidos en el notebook fueron los siguientes:

Algoritmo	Tiempo de ejecución aproximado
insertion_sort	0.11008350001065992 segundos
merge_sort	0.02439479998844974 segundos
sorted()	0.002694999988307245 segundos

Los resultados muestran que `insertion_sort` fue la implementación más lenta, lo cual es coherente con su complejidad temporal $O(n^2)$. En contraste, `merge_sort` presentó un mejor rendimiento, asociado a su complejidad esperada $O(n \log n)$. Finalmente, `sorted()` obtuvo el menor tiempo de ejecución, debido a que corresponde a una función optimizada internamente en Python. Esta comparación confirma que la selección del algoritmo influye directamente en el rendimiento del sistema.

El algoritmo `merge_sort` presentó un mejor desempeño que `insertion_sort`. Su estrategia recursiva divide la lista en partes más pequeñas, ordena cada subconjunto y luego combina los resultados. Desde el punto de vista teórico, su complejidad temporal esperada es $O(n \log n)$, lo que explica su mejor comportamiento frente a un algoritmo cuadrático cuando aumenta el número de registros analizados.

Por su parte, la función `sorted()` de Python obtuvo el mejor tiempo de ejecución. Esta función está altamente optimizada e implementada internamente para trabajar de manera eficiente con distintos escenarios de ordenamiento. Su inclusión permitió establecer una referencia práctica frente a las implementaciones manuales desarrolladas con fines académicos.

La interpretación de estos resultados permite distinguir entre valor pedagógico y eficiencia práctica. `insertion_sort` es útil para comprender la lógica básica de ordenamiento mediante estructuras de control, pero no resulta recomendable para volúmenes grandes de datos. `merge_sort`, en cambio, permite evidenciar una mejora algorítmica relevante mediante recursividad y división funcional. Finalmente, `sorted()` representa la alternativa más adecuada para un entorno de análisis real, ya que entrega mejor rendimiento, menor complejidad operativa para el usuario y mayor confiabilidad al estar incorporada en el lenguaje.

Desde la perspectiva de optimización, la mejora principal consistió en comparar una implementación estructurada simple con una implementación recursiva más eficiente y con una función optimizada del lenguaje. Esta comparación permitió demostrar que no basta con que un algoritmo funcione correctamente; también es necesario evaluar su rendimiento cuando se aplica sobre datos reales.

En relación con la complejidad espacial, el análisis desarrollado se centra principalmente en una

comparación teórica entre las implementaciones evaluadas. `insertion_sort` tiende a utilizar menos memoria adicional, ya que opera principalmente sobre una copia de la lista de entrada y realiza desplazamientos internos durante el proceso de ordenamiento. En cambio, `merge_sort` requiere memoria adicional para dividir la lista original en sublistas y combinar posteriormente los resultados, lo que implica un mayor uso de espacio durante la ejecución recursiva.

Por su parte, `sorted()` corresponde a una función optimizada del lenguaje Python, por lo que su gestión interna de memoria depende de la implementación del intérprete y de las estructuras utilizadas durante el ordenamiento. Aunque la medición empírica principal realizada en el notebook se centró en la complejidad temporal mediante `timeit`, esta comparación espacial permite complementar la evaluación técnica considerando también el uso esperado de memoria.

Desde esta perspectiva, la elección final depende del contexto de uso. Para fines pedagógicos, `insertion_sort` permite comprender con claridad la lógica estructurada del ordenamiento, aunque presenta bajo rendimiento temporal. `merge_sort` ofrece una mejora importante en tiempo de ejecución gracias a su estrategia recursiva, pero utiliza más memoria auxiliar. En un escenario práctico de ciencia de datos, `sorted()` representa la alternativa más eficiente y recomendable, debido a su rendimiento observado y a su optimización interna.

En síntesis, el análisis de eficiencia permite concluir que la solución más adecuada para un flujo real de análisis de datos es `sorted()`, mientras que `merge_sort` funciona como una alternativa algorítmica relevante para demostrar recursividad y mejora frente a una solución cuadrática. La comparación entre las tres implementaciones evidencia la importancia de evaluar tanto el tiempo de ejecución como el uso esperado de memoria antes de seleccionar una estrategia algorítmica.

La medición de tiempos mediante `timeit` permite comparar implementaciones de manera reproducible, ya que esta herramienta está diseñada para evaluar fragmentos de código bajo condiciones controladas (Python Software Foundation, 2026).

II.e Diseño estructurado del código

El diseño estructurado del código desarrollado en la Fase 3 se organizó con el propósito de mantener un flujo claro, modular y escalable entre la carga de datos, el procesamiento algorítmico, la medición de eficiencia y la documentación de resultados. Esta organización permite que cada componente del proyecto cumpla una responsabilidad específica, facilitando la comprensión, la reutilización y el mantenimiento del código.

La arquitectura general del trabajo se distribuye entre tres elementos principales: el notebook `F3_algoritmos_complejidad.ipynb`, el script `algoritmos_f3.py` y la carpeta de resultados `F3/results`. El notebook actúa como espacio de ejecución, análisis e interpretación; el script concentra clases y métodos reutilizables; y la carpeta de resultados almacena las salidas generadas durante la medición de complejidad.

La estructura funcional puede representarse de la siguiente manera:

```

F3/
├── notebooks/
│   └── F3_algoritmos_complejidad.ipynb
├── src/
│   └── algoritmos_f3.py
├── results/
│   └── mediciones_complejidad_f3.txt
└── README_F3.md

```

Esta separación favorece una organización coherente con buenas prácticas de proyectos de ciencia de datos. Los notebooks se utilizan para documentar y ejecutar el análisis, los scripts almacenan lógica reutilizable y los archivos de resultados permiten conservar evidencia del proceso ejecutado.

El flujo de datos implementado sigue una secuencia progresiva y verificable:

Carga del dataset procesado



Exploración inicial



Validación básica de integridad



Selección de variable numérica



Transformación a lista de trabajo



Aplicación de algoritmos



Medición de tiempos



Comparación de resultados



Exportación de evidencia

Esta estructura permite mantener claridad entre las entradas, las transformaciones y las salidas. La entrada corresponde al dataset `ai_job_impact_clean.csv`, generado en la Fase 2. El procesamiento se realiza mediante funciones y clases en Python, mientras que las salidas se reflejan en el notebook y en el archivo `mediciones_complejidad_f3.txt`.

Desde el punto de vista de cohesión, cada componente cumple una función concreta. El notebook integra la narrativa técnica y la ejecución de celdas; el script `algoritmos_f3.py` agrupa la lógica asociada al preprocesamiento y análisis mediante clases; las funciones `insertion_sort`, `merge_sort` y `combinar_listas` se encargan específicamente de resolver tareas de ordenamiento; y la librería `timeit` se utiliza exclusivamente para medir eficiencia temporal.

Esta separación de responsabilidades reduce el acoplamiento entre componentes. Por ejemplo, el notebook puede importar clases desde `algoritmos_f3.py` sin concentrar toda la lógica dentro de sus celdas. Del mismo modo, los algoritmos de ordenamiento reciben listas como entrada, por lo que no dependen directamente del DataFrame completo. Esta decisión mejora la reutilización del código, ya que los algoritmos pueden aplicarse a otras variables numéricas si se requiere ampliar el análisis.

La recursividad se incorpora mediante el algoritmo `merge_sort`. Este algoritmo divide la lista original en subconjuntos más pequeños hasta alcanzar un caso base, y luego combina los resultados mediante la función auxiliar `combinar_listas`. Esta división funcional permite resolver el problema de ordenamiento de manera más eficiente que un algoritmo estructurado simple como `insertion_sort`, manteniendo una lógica clara y coherente con el análisis de complejidad desarrollado en la sección anterior.

El diseño también incorpora una primera aproximación a la Programación Orientada a Objetos mediante las clases `PreprocesadorAIJob` y `PreprocesadorAIJobAvanzado`. La clase base se encarga de cargar, explorar, limpiar, validar y resumir el dataset, mientras que la clase avanzada extiende esa funcionalidad mediante métodos de análisis específicos. Esta relación permite separar responsabilidades y proyectar una evolución hacia una arquitectura más robusta y reutilizable.

La organización modular inicial puede resumirse en la siguiente tabla:

Componente	Responsabilidad
F3_algoritmos_complejidad.ipynb	Ejecutar el flujo de Fase 3, documentar decisiones y mostrar resultados.
algoritmos_f3.py	Contener clases y métodos reutilizables para administrar el dataset.
PreprocesadorAIJob	Cargar, explorar, limpiar, validar y resumir el dataset.
PreprocesadorAIJobAvanzado	Extender la clase base con métodos de análisis salarial y filtros.
insertion_sort	Ordenar datos mediante programación estructurada.
merge_sort	Ordenar datos mediante programación recursiva.
combinar_listas	Integrar dos listas ordenadas dentro del algoritmo recursivo.
timeit	Medir tiempos de ejecución de las implementaciones.
mediciones_complejidad_f3.txt	Registrar resultados de eficiencia temporal.

En términos de escalabilidad, el diseño actual permite incorporar nuevas funcionalidades sin modificar completamente el flujo principal. Por ejemplo, podrían agregarse nuevas estrategias de ordenamiento, funciones de búsqueda, algoritmos de clasificación, mediciones de memoria o clases especializadas para análisis por industria, nivel de riesgo o adopción de inteligencia artificial.

Asimismo, el diseño puede evolucionar hacia una arquitectura más robusta mediante la separación de algoritmos en un módulo específico, por ejemplo `ordenadores.py`, y la incorporación de una interfaz común para estrategias de ordenamiento. Esta mejora permitiría aplicar polimorfismo de forma explícita, ya que distintas clases podrían implementar un mismo método, como `ordenar()`, manteniendo bajo acoplamiento entre el notebook y las implementaciones concretas.

Aunque la implementación actual ya presenta una separación básica de responsabilidades, se reconoce que existen oportunidades de mejora para fortalecer el diseño arquitectónico. Entre ellas se encuentran la incorporación de pruebas automatizadas, el manejo explícito de excepciones, la medición formal de complejidad espacial y la implementación de una familia de clases polimórficas para los algoritmos de ordenamiento.

En síntesis, el diseño estructurado del código permite organizar el desarrollo de la Fase 3 de manera clara, modular y trazable. La separación entre notebook, script y resultados facilita la comprensión del flujo de trabajo; la implementación de funciones estructuradas y recursivas permite comparar estrategias algorítmicas; y la incorporación inicial de clases entrega una base adecuada para continuar evolucionando hacia un sistema más escalable y mantenible.

III. Implementación de código modular y robusto

La implementación de código modular y robusto en la Fase 3 se orienta a mejorar la organización del proyecto mediante el uso de clases, métodos y separación de responsabilidades. Esta etapa busca avanzar desde un flujo principalmente procedimental hacia una estructura más reutilizable, mantenible y coherente con los principios de la Programación Orientada a Objetos.

El desarrollo se apoya en el script `algoritmos_f3.py`, ubicado en la carpeta `F3/src`, y en el notebook `F3_algoritmos_complejidad.ipynb`, ubicado en `F3/notebooks`. El script contiene las clases encargadas de administrar y analizar el dataset, mientras que el notebook ejecuta el flujo completo, documenta las decisiones técnicas y presenta los resultados obtenidos.

La implementación modular permite separar la lógica de procesamiento de datos, el análisis algorítmico, la medición de eficiencia y la documentación de resultados. Esta separación contribuye a mejorar la trazabilidad del proyecto y facilita su extensión en futuras fases.

III.a Programación orientada a objetos

La Programación Orientada a Objetos se incorporó en la Fase 3 mediante la creación de clases orientadas a administrar el dataset **AI Job Impact** y extender sus funcionalidades de análisis. Esta implementación permite encapsular operaciones relacionadas con la carga, exploración, limpieza, transformación, validación y resumen del conjunto de datos, reduciendo la repetición de instrucciones dentro del notebook y favoreciendo una estructura más modular, mantenible y reutilizable.

El archivo `algoritmos_f3.py` contiene la clase base `PreprocesadorAIJob`, cuya responsabilidad principal es administrar el dataset utilizado en el proyecto. Esta clase recibe como parámetro la ruta del archivo CSV y almacena internamente el `DataFrame` cargado para ser utilizado por sus métodos. A diferencia de una implementación completamente pública, la versión actual utiliza atributos protegidos `_ruta` y `_df`, lo que fortalece el encapsulamiento del estado interno del objeto.

La estructura general de la clase base puede resumirse de la siguiente manera:

```
class PreprocesadorAIJob:
    def __init__(self, ruta_csv):
        self._ruta = ruta_csv
        self._df = None

    @property
    def ruta(self):
        return self._ruta
```

```
@property
def df(self):
    return self._df
```

El uso de atributos protegidos permite indicar que la ruta del archivo y el DataFrame cargado forman parte del estado interno de la clase. Estos atributos no se manipulan directamente desde el flujo principal del notebook, sino que se administran mediante propiedades y métodos específicos. Esta decisión mejora el control sobre los datos, reduce la exposición directa del estado interno y contribuye a la mantenibilidad del código.

La clase Preprocesador AIJob implementa métodos asociados al ciclo básico de procesamiento del dataset. Entre ellos se encuentran cargar(), explorar(), limpiar(), transformar(), validar(), obtener_salarios() y resumen(). Cada método cumple una responsabilidad específica dentro del flujo de trabajo, lo que favorece la cohesión de la clase y evita mezclar operaciones no relacionadas dentro de una misma función.

El método cargar() permite leer el archivo CSV desde la ruta definida y almacenar el resultado en el atributo protegido _df. Posteriormente, el método explorar() permite revisar los primeros registros, la información general y la presencia de valores nulos. El método limpiar() elimina registros con valores faltantes, mientras que validar() verifica que el dataset no contenga nulos después del procesamiento.

Una mejora relevante incorporada en esta fase corresponde al método transformar(), el cual aplica conversión explícita de columnas numéricas mediante pd.to_numeric(..., errors="coerce"). Este método permite asegurar que variables como salary_before_ai, salary_after_ai, age, years_experience, work_hours_per_week y job_satisfaction queden en formatos adecuados para análisis cuantitativos, ordenamientos y estadísticas descriptivas.

La estructura funcional de este método puede representarse de la siguiente manera:

```
def transformar(self):
    columnas_numericas = [
        "salary_before_ai",
        "salary_after_ai",
        "age",
        "years_experience",
        "work_hours_per_week",
        "job_satisfaction"
    ]

    for columna in columnas_numericas:
        if columna in self._df.columns:
            self._df[columna] = pd.to_numeric(
                self._df[columna],
                errors="coerce"
            )

    self._df = self._df.dropna()
    return self._df
```

Esta transformación fortalece la robustez del flujo de datos, ya que convierte a valores nulos aquellos

registros que no puedan interpretarse como numéricos y luego elimina inconsistencias mediante `dropna()`. De esta manera, el dataset queda preparado para ser utilizado por los algoritmos y métodos analíticos de la Fase 3.

Además de la clase base, se implementó la clase `PreprocesadorAIJobAvanzado`, la cual hereda de `PreprocesadorAIJob` e incorpora funcionalidades adicionales. Esta clase reutiliza la inicialización de la clase base mediante `super().__init__(ruta_csv)`, evitando duplicación de código y manteniendo una relación clara entre una clase general y una clase especializada.

La relación de herencia puede representarse de la siguiente manera:

```
class PreprocesadorAIJobAvanzado(PreprocesadorAIJob):
    def __init__(self, ruta_csv):
        super().__init__(ruta_csv)
```

Esta estructura evidencia herencia, ya que la clase avanzada conserva las capacidades de la clase base y agrega nuevos métodos orientados al análisis del dataset. Entre estos métodos se encuentran `promedio_salario()`, `salario_maximo()`, `salario_minimo()`, `filtrar_por_riesgo()` y `filtrar_por_industria()`. Estos métodos permiten realizar cálculos y filtros específicos sin modificar la clase base, favoreciendo la extensibilidad del sistema.

El polimorfismo se evidencia mediante la sobrescritura del método `resumen()` en la clase `PreprocesadorAIJobAvanzado`. Mientras la clase base `PreprocesadorAIJob` define un método `resumen()` para entregar estadísticas descriptivas generales del dataset, la clase avanzada redefine este mismo método para incorporar un comportamiento adicional, mostrando un encabezado de resumen avanzado y luego reutilizando la funcionalidad original mediante `super().resumen()`.

El fragmento correspondiente es el siguiente:

```
def resumen(self):
    print("\n=== Resumen avanzado del dataset ===\n")
    return super().resumen()
```

Esta implementación permite que un mismo método tenga comportamientos distintos según la clase desde la cual se invoque. De esta manera, el código demuestra polimorfismo por sobrescritura de métodos, manteniendo una relación clara entre clase base y clase derivada. Además, el uso de `super()` evita duplicación de código y permite extender la funcionalidad original sin modificar directamente la clase base.

Desde el punto de vista de cohesión, ambas clases mantienen responsabilidades relacionadas con la administración y análisis del dataset. `PreprocesadorAIJob` concentra las tareas generales de carga, exploración, limpieza, transformación, validación y resumen, mientras que `PreprocesadorAIJobAvanzado` amplía el análisis mediante operaciones estadísticas y filtros específicos. Esta organización evita mezclar responsabilidades no relacionadas dentro de una misma clase.

En cuanto al acoplamiento, el diseño mantiene una dependencia controlada entre el notebook y el script. El notebook importa las clases desde `algoritmos_f3.py` y las utiliza para ejecutar operaciones sobre el dataset, pero la lógica principal se mantiene centralizada en el módulo externo. Esta separación permite que el código pueda mantenerse, revisarse y reutilizarse sin depender completamente de las celdas del notebook.

La estructura orientada a objetos puede resumirse en la siguiente tabla:

Clase	Tipo	Responsabilidad principal
PreprocesadorAIJob	Clase base	Cargar, explorar, limpiar, transformar, validar y resumir el dataset.
PreprocesadorAIJobAvanzado	Clase hija	Extender el análisis con estadísticas salariales, filtros y sobrescritura de métodos.

Los métodos implementados permiten organizar el código de manera clara y reutilizable:

Método	Clase	Propósito
__init__()	Ambas clases	Inicializar la ruta del dataset y preparar el estado interno del objeto.
ruta	PreprocesadorAIJob	Propiedad que devuelve la ruta del dataset.
df	PreprocesadorAIJob	Propiedad que devuelve el DataFrame cargado.
cargar()	PreprocesadorAIJob	Leer el dataset desde un archivo CSV.
explorar()	PreprocesadorAIJob	Mostrar información general del dataset.
limpiar()	PreprocesadorAIJob	Eliminar registros con valores nulos.
transformar()	PreprocesadorAIJob	Convertir columnas numéricas y depurar inconsistencias.
validar()	PreprocesadorAIJob	Comprobar ausencia de valores nulos.
obtener_salarios()	PreprocesadorAIJob	Extraer la columna salary_after_ai como lista.
resumen()	Ambas clases	Generar estadísticas descriptivas, con sobrescritura en la clase avanzada.
promedio_salario()	PreprocesadorAIJobAvanzado	Calcular salario promedio posterior a IA.
salario_maximo()	PreprocesadorAIJobAvanzado	Obtener salario máximo posterior a IA.
salario_minimo()	PreprocesadorAIJobAvanzado	Obtener salario mínimo posterior a IA.
filtrar_por_riesgo()	PreprocesadorAIJobAvanzado	Filtrar registros por riesgo de automatización.
filtrar_por_industria()	PreprocesadorAIJobAvanzado	Filtrar registros por industria.

En síntesis, la implementación orientada a objetos desarrollada en la Fase 3 permite organizar el procesamiento del dataset mediante clases, métodos, encapsulamiento, herencia y polimorfismo. Esta estructura mejora la modularidad del proyecto, reduce la repetición de código en el notebook y entrega una base adecuada para futuras ampliaciones. La incorporación de atributos protegidos, propiedades de acceso, método de transformación y sobrescritura de resumen() fortalece el cumplimiento de los principios fundamentales de la Programación Orientada a Objetos.

La organización mediante clases, métodos y herencia responde a los principios fundamentales de la Programación Orientada a Objetos, los cuales permiten mejorar la modularidad, reutilización y mantenibilidad del código (Salinas, 2026b).

III.b Documentación de arquitectura y funcionalidad del código

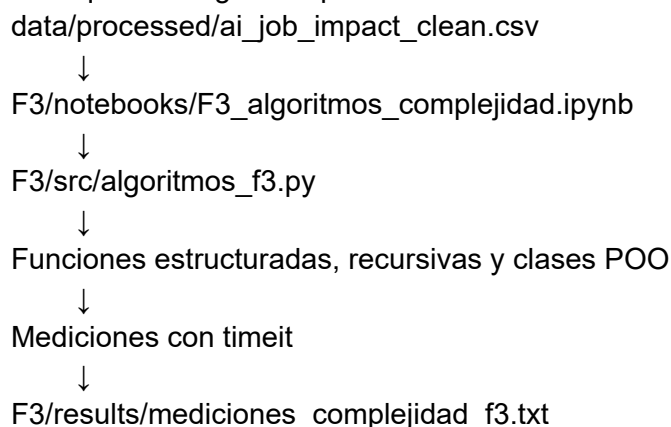
La documentación de arquitectura y funcionalidad del código se desarrolló con el propósito de explicar de manera clara cómo se organizan los componentes técnicos de la Fase 3, qué función cumple cada archivo dentro del proyecto y cuáles fueron las decisiones adoptadas para construir un flujo de trabajo reproducible, modular y trazable.

El desarrollo técnico de esta fase se organiza principalmente en torno al notebook `F3_algoritmos_complejidad.ipynb`, el script `algoritmos_f3.py`, la carpeta `F3/results` y el archivo `README_F3.md`. Cada uno de estos componentes cumple una función específica dentro de la arquitectura del proyecto.

Componente	Función dentro de la arquitectura
F3_algoritmos_complejidad.ipynb	Ejecutar el flujo técnico de la Fase 3, integrar explicaciones en Markdown y mostrar resultados intermedios y finales.
algoritmos_f3.py	Almacenar funciones, clases y métodos reutilizables relacionados con el análisis algorítmico y el procesamiento del dataset.
F3/results	Registrar los resultados generados por las mediciones de complejidad temporal.
README_F3.md	Documentar instrucciones de ejecución, estructura de carpetas, dependencias y propósito de la fase.
ai_job_impact_clean.csv	Dataset procesado en Fase 2 utilizado como entrada principal para el análisis algorítmico.

Esta distribución permite separar la ejecución, la lógica reutilizable, la documentación y los resultados. El notebook no concentra toda la implementación en celdas aisladas, sino que se apoya en un script externo para utilizar funciones, clases y métodos reutilizables. Esta decisión permite mejorar la mantenibilidad del código y facilita su revisión por otros integrantes del equipo o por el evaluador.

La arquitectura general puede resumirse mediante el siguiente flujo:



La decisión de reutilizar el dataset `ai_job_impact_clean.csv` responde a la continuidad metodológica entre fases. En la Fase 2 se desarrolló el pipeline de limpieza y transformación de datos, mientras que en la Fase 3 el foco se desplaza hacia la construcción algorítmica, la medición de eficiencia y la incorporación de

principios de Programación Orientada a Objetos. Esta separación evita duplicar procesos y permite mantener una trazabilidad clara entre los datos preparados y los algoritmos aplicados.

Desde el punto de vista del diseño algorítmico, se incorporaron tres estrategias de ordenamiento: `insertion_sort`, `merge_sort` y `sorted()`. La primera permite representar una solución estructurada basada en ciclos y condicionales; la segunda permite aplicar recursividad mediante división funcional; y la tercera actúa como referencia optimizada del lenguaje Python. Esta comparación permite analizar no solo si una solución funciona, sino también cuál es su desempeño relativo frente a alternativas más eficientes.

Un fragmento representativo del algoritmo estructurado es el siguiente:

```
def insertion_sort(lista):
    """
    Ordena una lista utilizando el algoritmo Insertion Sort.
    """

    lista = lista.copy()

    for i in range(1, len(lista)):
        clave = lista[i]
        j = i - 1

        while j >= 0 and lista[j] > clave:
            lista[j + 1] = lista[j]
            j -= 1
        lista[j + 1] = clave
    return lista
```

Este algoritmo fue incluido porque permite evidenciar programación estructurada mediante ciclos, condicionales y control secuencial del flujo. Aunque su complejidad temporal puede ser $O(n^2)$, resulta útil como referencia pedagógica para comparar con algoritmos más eficientes.

Por su parte, el algoritmo `merge_sort` fue incorporado para evidenciar recursividad y división del problema en subproblemas más pequeños:

```
def merge_sort(lista):
    """
    Implementación recursiva del algoritmo Merge Sort.
    """

    if len(lista) <= 1:
        return lista
    mitad = len(lista) // 2
    izquierda = merge_sort(lista[:mitad])
    derecha = merge_sort(lista[mitad:])

    return combinar_listas(izquierda, derecha)
```

La elección de `merge_sort` se justifica porque permite observar cómo una estrategia recursiva puede mejorar el desempeño frente a un algoritmo estructurado simple. Su complejidad temporal esperada $O(n \log n)$ lo convierte en una alternativa más eficiente para volúmenes de datos mayores.

La función `sorted()` fue utilizada como punto de comparación porque representa una solución optimizada incorporada en Python. Su inclusión permite contrastar algoritmos implementados manualmente con una herramienta eficiente del lenguaje, fortaleciendo el análisis técnico y la justificación de decisiones.

En relación con la Programación Orientada a Objetos, el script `algoritmos_f3.py` incorpora la clase `PreprocesadorAIJob`, encargada de administrar el dataset mediante métodos de carga, exploración, limpieza, transformación, validación y resumen.

También se implementa la clase `PreprocesadorAIJobAvanzado`, que hereda de la clase base e incorpora operaciones adicionales de análisis.

Un fragmento representativo de la estructura orientada a objetos es el siguiente:

```
class PreprocesadorAIJob:
    def __init__(self, ruta_csv):
        self._ruta = ruta_csv
        self._df = None

    @property
    def ruta(self):
        return self._ruta

    @property
    def df(self):
        return self._df
```

Esta estructura evidencia encapsulamiento mediante el uso de atributos protegidos `_ruta` y `_df`, los cuales representan el estado interno del objeto. En lugar de manipular directamente estos atributos desde el notebook, el acceso se realiza mediante propiedades y métodos específicos. Esta decisión mejora el control sobre los datos, reduce la exposición directa del estado interno y fortalece la mantenibilidad del código.

Una mejora relevante dentro de la arquitectura corresponde al método `transformar()`, incorporado para convertir columnas numéricas mediante `pd.to_numeric(..., errors="coerce")`. Esta transformación permite asegurar que variables como `salary_before_ai`, `salary_after_ai`, `age`, `years_experience`, `work_hours_per_week` y `job_satisfaction` se encuentren en formatos adecuados para cálculos, ordenamientos y estadísticas descriptivas.

```
def transformar(self):
    columnas_numericas = [
        "salary_before_ai",
        "salary_after_ai",
        "age",
        "years_experience",
```

```

        "work_hours_per_week",
        "job_satisfaction"
    ]

    for columna in columnas_numericas:
        if columna in self._df.columns:
            self._df[columna] = pd.to_numeric(
                self._df[columna],
                errors="coerce"
            )

    self._df = self._df.dropna()

    return self._df

```

El uso de `errors="coerce"` permite controlar registros que no puedan interpretarse como numéricos, convirtiéndolos en valores nulos. Posteriormente, `dropna()` elimina dichas inconsistencias. Esta decisión fortalece la robustez del procesamiento antes de ejecutar algoritmos y mediciones de eficiencia.

La clase avanzada extiende el comportamiento de la clase base:

```

class PreprocesadorAIJobAvanzado(PreprocesadorAIJob):
    def __init__(self, ruta_csv):
        super().__init__(ruta_csv)

    def resumen(self):
        print("\n=== Resumen avanzado del dataset ===\n")
        return super().resumen()

```

Esta decisión evidencia el uso de herencia y polimorfismo. La herencia se observa en la relación entre `PreprocesadorAIJobAvanzado` y `PreprocesadorAIJob`, mientras que el polimorfismo se demuestra mediante la sobrescritura del método `resumen()`. La clase avanzada redefine el comportamiento del método y, al mismo tiempo, reutiliza la funcionalidad original mediante `super().resumen()`. De esta manera, un mismo método puede comportarse de forma distinta dependiendo de la clase desde la cual se invoque.

La documentación también incorpora celdas Markdown dentro del notebook, donde se explican las etapas del proceso, la función de cada bloque de código, las mediciones obtenidas y la interpretación de resultados. Esta combinación de código y explicación permite que el notebook funcione no solo como herramienta de ejecución, sino también como documento técnico reproducible.

La validación del código también fue documentada dentro del notebook. Además de la revisión de nulos, duplicados y tipos de datos, se incorporaron pruebas formales mediante `assert`, las cuales permiten comprobar el comportamiento de los algoritmos frente a listas vacías, listas con un único elemento y listas desordenadas. También se incorporó manejo de excepciones mediante `try/except`, fortaleciendo la capacidad del flujo para responder ante entradas no válidas sin interrumpir la ejecución general.

El avance de la Fase 3 puede entenderse como una evolución progresiva del proyecto:

Fase	Avance técnico
Fase 1	Definición de problemática, objetivos, entorno reproducible y estructura inicial del repositorio.
Fase 2	Desarrollo del pipeline de obtención, limpieza, transformación y validación del dataset.
Fase 3	Implementación de algoritmos, medición de eficiencia temporal, validaciones formales y aplicación de POO.

Esta evolución demuestra continuidad entre las fases y permite verificar que el desarrollo no corresponde a ejercicios aislados, sino a un proyecto progresivo donde cada etapa se apoya en los resultados de la anterior.

Desde el punto de vista de fuentes utilizadas, las decisiones técnicas se apoyan en los contenidos docentes de la Fase 3 sobre programación estructurada, recursividad y Programación Orientada a Objetos. También se consideran referencias técnicas asociadas al uso de Python, Pandas y timeit, dado que estas herramientas fueron utilizadas para cargar datos, manipular estructuras tabulares y medir tiempos de ejecución.

En síntesis, la documentación de arquitectura y funcionalidad permite comprender cómo se organiza el proyecto, qué rol cumple cada componente, por qué se eligieron determinados algoritmos y cómo se integra la Programación Orientada a Objetos dentro del flujo de trabajo. Esta documentación fortalece la trazabilidad, facilita la revisión del código y deja una base clara para futuras mejoras, como incorporar nuevas estrategias algorítmicas, ampliar las mediciones de desempeño y extender las clases existentes con nuevas funcionalidades analíticas.

IV. Repositorio GitHub (F3)

El repositorio GitHub constituye el espacio central de almacenamiento, control de versiones y trazabilidad del proyecto. Durante la Fase 3, su uso permitió organizar el núcleo algorítmico desarrollado, registrar los cambios aplicados sobre el código, documentar la estructura de trabajo y conservar evidencia del avance realizado por el equipo.

El repositorio colaborativo del proyecto se encuentra disponible en:

https://github.com/Prod1982/mcdi500_s1_grupo6

La utilización de GitHub permite mantener continuidad con las fases anteriores del proyecto. En la Fase 1 se configuró la estructura inicial del repositorio y el entorno reproducible; en la Fase 2 se incorporó el pipeline de obtención, limpieza y transformación de datos; y en la Fase 3 se agregaron los componentes asociados al núcleo algorítmico, las mediciones de complejidad y la implementación inicial de Programación Orientada a Objetos.

La estructura correspondiente a la Fase 3 se encuentra organizada de la siguiente manera:

```
F3/
├── notebooks/
│   └── F3_algoritmos_complejidad.ipynb
├── src/
│   └── algoritmos_f3.py
├── results/
│   └── mediciones_complejidad_f3.txt
└── README_F3.md
```

Esta organización permite diferenciar claramente los componentes de la Fase 3 respecto de los archivos desarrollados en las fases anteriores. La carpeta notebooks contiene el notebook ejecutable principal de la fase; la carpeta src almacena el script con clases y métodos reutilizables; la carpeta results conserva los resultados de las mediciones de complejidad; y el archivo README_F3.md documenta el propósito, la estructura y la forma de ejecución del avance.

El repositorio también mantiene las carpetas y archivos generales del proyecto, incluyendo los datos originales y procesados, documentación, resultados, scripts de fases anteriores y archivos de configuración. Esta continuidad permite evidenciar que el trabajo de Fase 3 no se desarrolla de manera aislada, sino como una extensión del pipeline construido en Fase 2.

La estructura general del proyecto puede resumirse de la siguiente manera:

```
mcdi500_s1_grupo6/
├── data/
│   ├── raw/
│   │   └── ai_job_impact.csv
│   └── processed/
│       └── ai_job_impact_clean.csv
├── docs/
├── images/
├── notebooks/
├── results/
├── src/
├── F3/
│   ├── notebooks/
│   ├── src/
│   ├── results/
│   └── README_F3.md
├── README.md
└── requirements.txt
```

El archivo README_F3.md cumple una función relevante dentro del repositorio, ya que documenta las instrucciones de ejecución del notebook, la finalidad de la fase, los archivos involucrados, las dependencias necesarias y las funcionalidades implementadas. Esta documentación facilita que cualquier integrante del equipo o evaluador pueda comprender la estructura de la fase y ejecutar el notebook desde un entorno configurado.

Entre los elementos documentados en el README de Fase 3 se consideran:

- Descripción del objetivo de la Fase 3.
- Ubicación del notebook ejecutable.
- Ubicación del script algoritmos_f3.py.
- Descripción de los resultados generados.
- Instrucciones generales de ejecución.
- Relación con el dataset procesado en la Fase 2.
- Uso de algoritmos estructurados y recursivos.
- Transformación de columnas numéricas mediante `pandas.to_numeric()`.
- Validaciones posteriores al proceso de limpieza y transformación.
- Uso de clases orientadas a objetos.
- Aplicación de herencia y sobrescritura de métodos para demostrar polimorfismo.
- Generación de visualizaciones exploratorias y gráficos comparativos de tiempos

El archivo algoritmos_f3.py contiene los principales elementos técnicos reutilizables de la fase. En este módulo se implementan los algoritmos `insertion_sort`, `merge_sort` y la función auxiliar `combinar_listas`, además de las clases `PreprocesadorAIJob` y `PreprocesadorAIJobAvanzado`. La clase base incorpora métodos para cargar, explorar, limpiar, transformar, validar y resumir el dataset, mientras que la clase avanzada extiende su funcionalidad mediante herencia.

Una mejora relevante del repositorio es que el script incorpora encapsulamiento mediante atributos protegidos `_ruta` y `_df`, propiedades de acceso, transformación numérica mediante `pd.to_numeric(..., errors="coerce")` y polimorfismo mediante la sobrescritura del método `resumen()` en la clase `PreprocesadorAIJobAvanzado`. Esto permite evidenciar que el repositorio no solo contiene un notebook ejecutable, sino también un módulo reutilizable con una estructura orientada a objetos más robusta.

El historial de commits constituye una evidencia clave del proceso de desarrollo. A través de los commits realizados, es posible identificar la evolución del núcleo algorítmico, la incorporación de clases, la medición de complejidad, las mejoras del README y la actualización de documentación. Entre los cambios registrados durante esta fase se encuentran:

feat: crea estructura inicial fase 3
 feat: implementa pipeline de datos fase 2
 docs: actualiza README con instrucciones fase 2
 feat: agrega mediciones de complejidad fase 3
 Se agrega boxplot comparativo
 Modificación README_F3.md

Estos commits permiten demostrar que el repositorio fue actualizado progresivamente y que el trabajo se desarrolló mediante control de versiones. Además, evidencian la integración de scripts, resultados, documentación, visualizaciones y mejoras aplicadas durante el avance de la Fase 3.

Desde el punto de vista de trazabilidad, cada componente técnico del informe se encuentra respaldado por un archivo dentro del repositorio. El análisis algorítmico descrito en el informe se desarrolla en el notebook `F3_algoritmos_complejidad.ipynb`; las funciones y clases orientadas a objetos se encuentran en `F3/src/algoritmos_f3.py`; los resultados de las mediciones se almacenan en `F3/results/mediciones_complejidad_f3.txt`; y la documentación de ejecución se encuentra en `F3/README_F3.md`.

La relación entre los componentes puede resumirse en la siguiente tabla:

Evidencia	Ubicación en el repositorio	Función
Notebook F3	<code>F3/notebooks/F3_algoritmos_complejidad.ipynb</code>	Ejecutar y documentar el flujo de la Fase 3.
Script POO	<code>F3/src/algoritmos_f3.py</code>	Contener clases, métodos y herencia.
Resultados	<code>F3/results/mediciones_complejidad_f3.txt</code>	Registrar mediciones de complejidad temporal.
README F3	<code>F3/README_F3.md</code>	Explicar estructura, propósito e instrucciones de ejecución.
Dataset procesado	<code>data/processed/ai_job_impact_clean.csv</code>	Proveer la entrada limpia generada en la Fase 2.

La organización del repositorio mantiene coherencia con prácticas profesionales de proyectos analíticos, ya que separa datos, notebooks, scripts, resultados y documentación. Esta separación facilita la comprensión del proyecto, permite reproducir el flujo de trabajo y reduce el riesgo de mezclar archivos con

responsabilidades distintas.

Respecto de las contribuciones individuales, el historial de GitHub permite identificar modificaciones realizadas por distintos integrantes del equipo. Esto fortalece la evidencia de trabajo colaborativo, ya que los commits quedan asociados a usuarios específicos y permiten reconstruir la evolución del proyecto. La existencia de commits relacionados con el notebook, el README, las visualizaciones y las mediciones de complejidad permite demostrar participación en diferentes aspectos del avance.

En síntesis, el repositorio GitHub cumple una función esencial en la Fase 3, ya que organiza la evidencia técnica, documenta la evolución del código, conserva resultados reproducibles y respalda la relación entre notebook, script, informe y datos procesados. Su estructura diferenciada para F3, junto con el README actualizado y el historial de commits, permite demostrar trazabilidad, colaboración y continuidad respecto de las fases anteriores del proyecto.

V. Notebooks ejecutables (F3)

El notebook ejecutable correspondiente a la Fase 3 constituye la principal evidencia técnica del avance desarrollado durante esta etapa del proyecto. Su función es integrar el dataset procesado en la Fase 2 con la implementación de algoritmos estructurados y recursivos, mediciones de complejidad temporal, validaciones técnicas, uso de Programación Orientada a Objetos y documentación mediante celdas Markdown.

El archivo principal de esta fase corresponde a: F3/notebooks/F3_algoritmos_complejidad.ipynb

Este notebook fue diseñado para ejecutar de manera ordenada el flujo técnico de la Fase 3, manteniendo correspondencia con el informe y con la estructura del repositorio GitHub. Su desarrollo permite demostrar que el trabajo realizado no se limita a una descripción teórica, sino que cuenta con evidencia ejecutable, resultados intermedios y salidas verificables.

La estructura general del notebook sigue una secuencia progresiva:

Presentación de la Fase 3



Importación de librerías



Definición de rutas del proyecto



Carga del dataset procesado de Fase 2



Exploración y validación básica del dataset



Transformación de columnas numéricas



Selección de variables para análisis algorítmico



Implementación de `insertion_sort`



Implementación de `merge_sort`



Comparación con `sorted()`



Validaciones con `assert`



Manejo de excepciones con `try/except`



Medición de tiempos con `timeit`



Uso de clases desde `algoritmos_f3.py`



Aplicación de herencia, encapsulamiento y polimorfismo



Visualización exploratoria



Exportación de resultados



Interpretación técnica

Esta organización permite mantener un flujo de trabajo coherente entre entradas, procesamiento y salidas. La entrada principal corresponde al archivo `ai_job_impact_clean.csv`, generado durante la Fase 2 y almacenado en la carpeta `data/processed`. A partir de este archivo, el notebook carga el dataset, verifica su estructura y selecciona la variable `salary_after_ai` para las pruebas algorítmicas.

El notebook incorpora programación estructurada mediante el algoritmo `insertion_sort`. Esta implementación utiliza ciclos, condicionales y desplazamientos sucesivos para ordenar una lista de salarios. Su inclusión permite evidenciar una solución funcional basada en estructuras de control básicas y facilita la comparación con alternativas más eficientes.

También se implementa programación recursiva mediante el algoritmo `merge_sort`. Esta función divide la lista de entrada en subconjuntos más pequeños, aplica llamadas recursivas y combina los resultados mediante la función auxiliar `combinar_listas`. Esta estrategia permite demostrar el uso de recursividad y división funcional dentro de un problema aplicado a datos reales del proyecto.

Además, el notebook compara ambas implementaciones con la función optimizada `sorted()` de Python. Esta comparación permite contextualizar el rendimiento de los algoritmos manuales frente a una solución incorporada en el lenguaje, aportando evidencia para el análisis de eficiencia.

Una mejora relevante del notebook corresponde a la incorporación de transformación explícita de columnas numéricas. A través del método `transformar()` de la clase `PreprocesadorAIJob`, definido en el script `algoritmos_f3.py`, se aplica `pd.to_numeric(..., errors="coerce")` sobre variables como `salary_before_ai`, `salary_after_ai`, `age`, `years_experience`, `work_hours_per_week` y `job_satisfaction`. Esta transformación permite asegurar que las variables utilizadas en cálculos, ordenamientos y estadísticas descriptivas mantengan tipos numéricos consistentes.

El notebook también incorpora validaciones formales mediante `assert`. Estas pruebas permiten verificar que los algoritmos `insertion_sort` y `merge_sort` funcionen correctamente en casos normales y casos límite, tales como listas vacías, listas con un único elemento y listas pequeñas desordenadas. La correcta ejecución de estas validaciones fortalece la confiabilidad del flujo algorítmico.

Además, se incorpora manejo básico de excepciones mediante bloques `try/except`. Esta práctica permite controlar situaciones no esperadas sin interrumpir la ejecución completa del notebook, mejorando la robustez del proceso y aportando evidencia de verificación técnica.

La medición de complejidad temporal se realiza mediante la librería `timeit`. En el notebook se ejecutan mediciones sobre una muestra de 500 registros de la variable `salary_after_ai`, comparando los tiempos de ejecución de `insertion_sort`, `merge_sort` y `sorted()`. Los resultados obtenidos permiten evaluar empíricamente el desempeño de cada implementación y justificar la elección de alternativas más eficientes para escenarios de análisis reales.

Entre los resultados registrados en el notebook se identifican mediciones aproximadas como:

Implementación	Tiempo aproximado
<code>insertion_sort</code>	0.11008350001065992 segundos
<code>merge_sort</code>	0.02439479998844974 segundos
<code>sorted()</code>	0.002694999988307245 segundos

Estos resultados evidencian diferencias significativas entre las implementaciones evaluadas. `insertion_sort` presenta el mayor tiempo de ejecución, `merge_sort` mejora el rendimiento mediante recursividad y `sorted()` obtiene el mejor desempeño por tratarse de una función optimizada del lenguaje.

El notebook también integra elementos de Programación Orientada a Objetos mediante la importación y uso del script `F3/src/algoritmos_f3.py`. En este archivo se definen las clases `PreprocesadorAIJob` y `PreprocesadorAIJobAvanzado`, utilizadas para cargar, explorar, limpiar, transformar, validar y analizar el dataset.

La clase `PreprocesadorAIJob` evidencia encapsulamiento mediante atributos protegidos `_ruta` y `_df`, junto con propiedades de acceso como `ruta` y `df`. Esta organización permite controlar el estado interno del objeto y evita la manipulación directa del `DataFrame` desde el flujo principal del notebook.

La clase `PreprocesadorAIJobAvanzado` hereda de `PreprocesadorAIJob`, reutilizando su inicialización mediante `super().__init__(ruta_csv)`. Además, demuestra polimorfismo mediante la sobrescritura del método `resumen()`, el cual incorpora un comportamiento adicional y luego reutiliza la funcionalidad original de la clase base mediante `super().resumen()`.

Esta integración permite que el notebook no concentre toda la lógica en celdas aisladas, sino que utilice un módulo externo con funciones, clases y métodos reutilizables. De esta manera, se mejora la organización del código y se fortalece la relación entre notebook, script y repositorio.

El notebook contiene además documentación técnica integrada mediante celdas Markdown. Estas celdas explican el propósito de cada bloque, la lógica de los algoritmos implementados, la selección de variables, las validaciones realizadas, las mediciones obtenidas y la interpretación de los resultados. Esta combinación de código y explicación permite que el archivo funcione como evidencia reproducible y como documento técnico de apoyo.

La ejecución del notebook genera resultados intermedios y finales, entre ellos:

- Confirmación de carga correcta del dataset.
- Visualización de dimensiones del dataset.
- Revisión de valores nulos.
- Revisión de registros duplicados.
- Revisión de tipos de datos.
- Transformación de variables numéricas.
- Listas de salarios originales y ordenadas.
- Validaciones con `assert`.

- Manejo de excepciones con try/except.
- Tiempos de ejecución de algoritmos.
- Comparación de desempeño entre implementaciones.
- Estadísticas descriptivas mediante clases.
- Evidencia de encapsulamiento, herencia y polimorfismo.
- Visualización exploratoria de salarios antes y después de la IA.
- Exportación del archivo mediciones_complejidad_f3.txt.

La generación del archivo mediciones_complejidad_f3.txt en la carpeta F3/results permite conservar evidencia externa al notebook. Este resultado fortalece la trazabilidad del proceso, ya que las mediciones no quedan únicamente visibles en las celdas, sino también registradas como archivo dentro del repositorio. La relación entre el notebook y el repositorio se mantiene de la siguiente manera:

Elemento	Ubicación	Función
Notebook principal	F3/notebooks/F3_algoritmos_complejidad.ipynb	Ejecutar y documentar el flujo de Fase 3.
Script de clases y algoritmos	F3/src/algoritmos_f3.py	Contener funciones, clases, transformación, herencia y polimorfismo.
Dataset procesado	data/processed/ai_job_impact_clean.csv	Proveer los datos limpios provenientes de Fase 2.
Resultados	F3/results/mediciones_complejidad_f3.txt	Registrar mediciones de complejidad temporal.
README F3	F3/README_F3.md	Explicar estructura e instrucciones de ejecución.

Desde el punto de vista de reproducibilidad, el notebook utiliza rutas relativas dentro del proyecto, lo que permite ejecutarlo desde la estructura del repositorio sin depender de rutas absolutas del equipo local. Además, el uso del archivo requirements.txt contribuye a mantener control sobre las dependencias necesarias para reproducir el entorno.

En síntesis, el notebook F3_algoritmos_complejidad.ipynb representa una evidencia ejecutable, documentada y trazable del avance de la Fase 3. Integra datos procesados de la Fase 2, algoritmos estructurados y recursivos, transformación de variables numéricas, validaciones formales, mediciones de complejidad temporal, uso de clases orientadas a objetos, resultados exportados y documentación técnica. Su correspondencia con el informe y con la estructura del repositorio permite demostrar coherencia, reproducibilidad y continuidad en el desarrollo del proyecto.

VI. Conclusiones

El desarrollo de la Fase 3 permitió consolidar el núcleo algorítmico del proyecto mediante la integración de programación estructurada, programación recursiva y principios de Programación Orientada a Objetos. A partir del dataset procesado en la Fase 2, se implementaron algoritmos de ordenamiento, se aplicaron mediciones de eficiencia y se organizaron componentes reutilizables dentro de una estructura modular y trazable.

Desde el punto de vista algorítmico, la comparación entre `insertion_sort`, `merge_sort` y `sorted()` permitió evidenciar que distintas estrategias pueden producir el mismo resultado funcional, pero con diferencias significativas de rendimiento. `insertion_sort` permitió representar una solución estructurada basada en ciclos y condicionales; `merge_sort` incorporó recursividad y división funcional; mientras que `sorted()` actuó como referencia optimizada del lenguaje Python. Las mediciones temporales realizadas con `timeit` demostraron la importancia de evaluar empíricamente el desempeño antes de seleccionar una solución para escenarios de análisis de datos.

En relación con el preprocesamiento, la Fase 3 reutilizó el dataset limpio generado en la Fase 2 y aplicó validaciones adicionales orientadas al análisis algorítmico. La revisión de valores nulos, duplicados, tipos de datos y transformación de variables numéricas mediante `pd.to_numeric()` permitió asegurar que las entradas utilizadas por los algoritmos fueran coherentes y reproducibles.

La implementación orientada a objetos permitió avanzar hacia un diseño más modular y mantenible. La clase `PreprocesadorAIJob` encapsula operaciones de carga, exploración, limpieza, transformación, validación y resumen del dataset mediante atributos protegidos y propiedades de acceso. La clase `PreprocesadorAIJobAvanzado` extiende la clase base mediante herencia y demuestra polimorfismo al sobrescribir el método `resumen()`, reutilizando parte del comportamiento original mediante `super().resumen()`.

Las validaciones incorporadas contribuyeron a comprobar el funcionamiento del código bajo distintos escenarios, incluyendo casos normales, casos límite y manejo controlado de excepciones. Las pruebas con `assert` permitieron verificar el comportamiento de los algoritmos frente a listas vacías, listas con un solo elemento y listas desordenadas, mientras que los bloques `try/except` permitieron controlar errores sin detener el flujo general del notebook.

En síntesis, la Fase 3 permitió transformar el proyecto desde un pipeline de datos hacia una estructura algorítmica más robusta, modular y documentada. El trabajo desarrollado evidencia continuidad con las fases anteriores, aplicación práctica de conceptos de programación para ciencia de datos y una base escalable para continuar incorporando nuevas funcionalidades analíticas en futuras etapas.

VII. Bibliografía

Referencias

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., Fernández del Río, J., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., & Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>

McKinney, W. (2023). *Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media.

Python Software Foundation. (2026). *timeit — Measure execution time of small code snippets*. Python Documentation. <https://docs.python.org/3/library/timeit.html>

Salinas, O. (2026a). *Fase 3: Desarrollo algorítmico, eficiencia y complejidad computacional* [Material docente]. Universidad Andrés Bello.

Salinas, O. (2026b). *Programación orientada a objetos aplicada a la ciencia de datos* [Material docente]. Universidad Andrés Bello.

The pandas development team. (2026). *pandas documentation*. pandas. <https://pandas.pydata.org/docs/>
 Universidad Andrés Bello. (2026). *Guía de apoyo Sumativa 3 Fase 3: Núcleo algorítmico, eficiencia y aplicación de principios de programación orientada a objetos* [Guía de evaluación]. Universidad Andrés Bello.